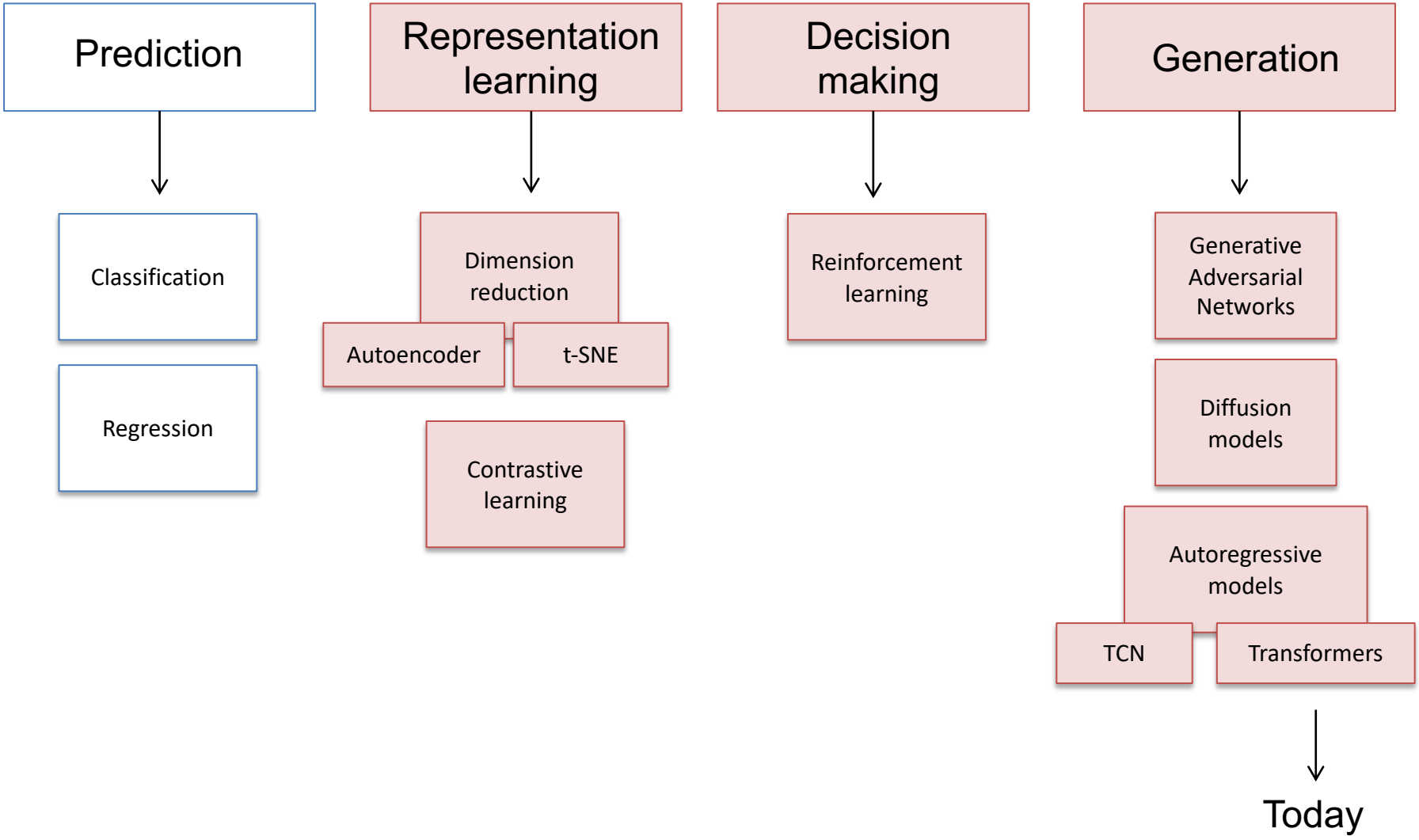
# Machine Intelligence:: Deep Learning
# Week 7: Vision Transformers

*Beate Sick, Lilach Goren Huber, Pascal Bühler*

Institut für Datenanalyse und Prozessdesign
Zürcher Hochschule für Angewandte Wissenschaften

# Last time


Prediction
Classification
Regression
Representation learning
Dimension reduction
Autoencoder
t-SNE
Contrastive learning
Decision making
Reinforcement learning
Generation
Generative Adversarial Networks
Diffusion models
Autoregressive models
TCN
Transformers
Today

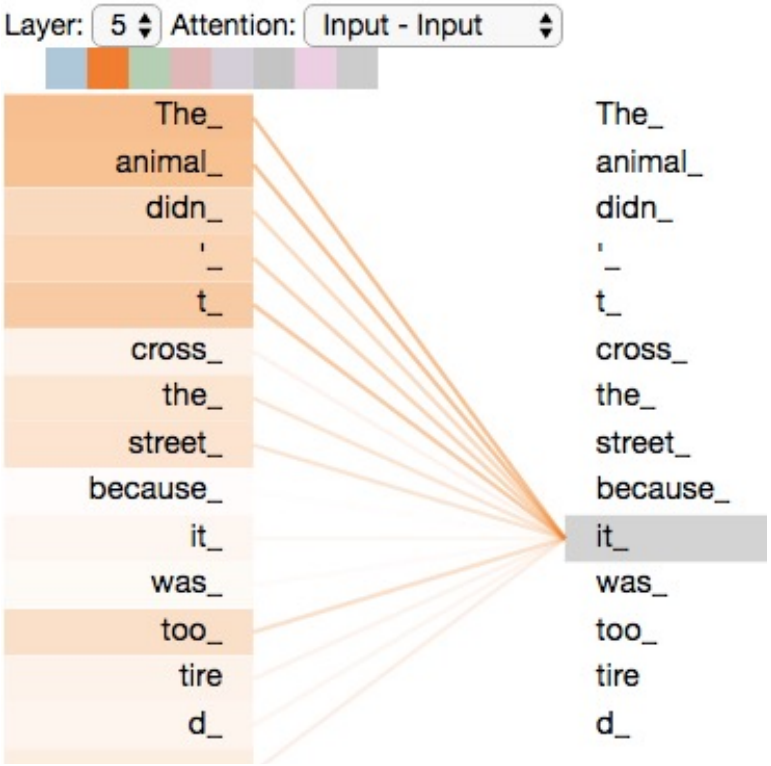
# Goal of today

- Last time: overview of generative models.
- One class of generative models: transformer-based models (e.g for text generation).
- Transformers are more general. can be used for:
  - Prediction: classification, segmentation, detection.
  - Representation learning.
  - Decision making.
  - Generation.

- Today: recap the idea of transformers & attention.
- Focus on vision transformers:
  - Why use them?
  - Basic principles
  - How to use?

# Transformers (Attention)

- Attention-based modeling.
- No convolutions, no recurrence — only **self-attention**.
- Every token attends to all others (via attention mask).




https://jalammar.github.io/illustrated-transformer/

# Why do we need vision transformers (ViT)?

- **Attention**: New paradigm extended from language to computer vision. The goal: learn strong meaningful representations with global and local context.

- **Language and vision** are the two big domains with successful DL applications. Merging them using one architecture allows for richer options ➔ multi-modal tasks (involving text, images, audio, video…)

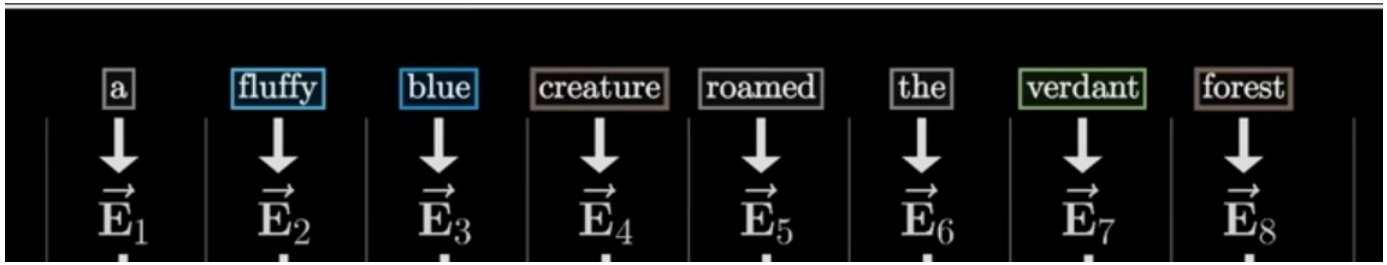
- **Allow more freedom in feature discovery:** ML methods provide diffferent levels of inductive bias during learning: handcrafted features ➔ many, CNNs➔ fewer, transformers ➔ yet fewer. It is the most data driven architecture.

- **Built-in interpretability** mechanism = attention maps.

- **Can it replace and even outperforms CNNs?**

# Text & Vision

- Transformers = attention = disruptive change in NLP




- The strength: embed meanings by identifying relationships between words:
  - Initial word (token) embedding
  - Calculate relationships with other words
  - Modify the embedding based on these relationships

bank of England

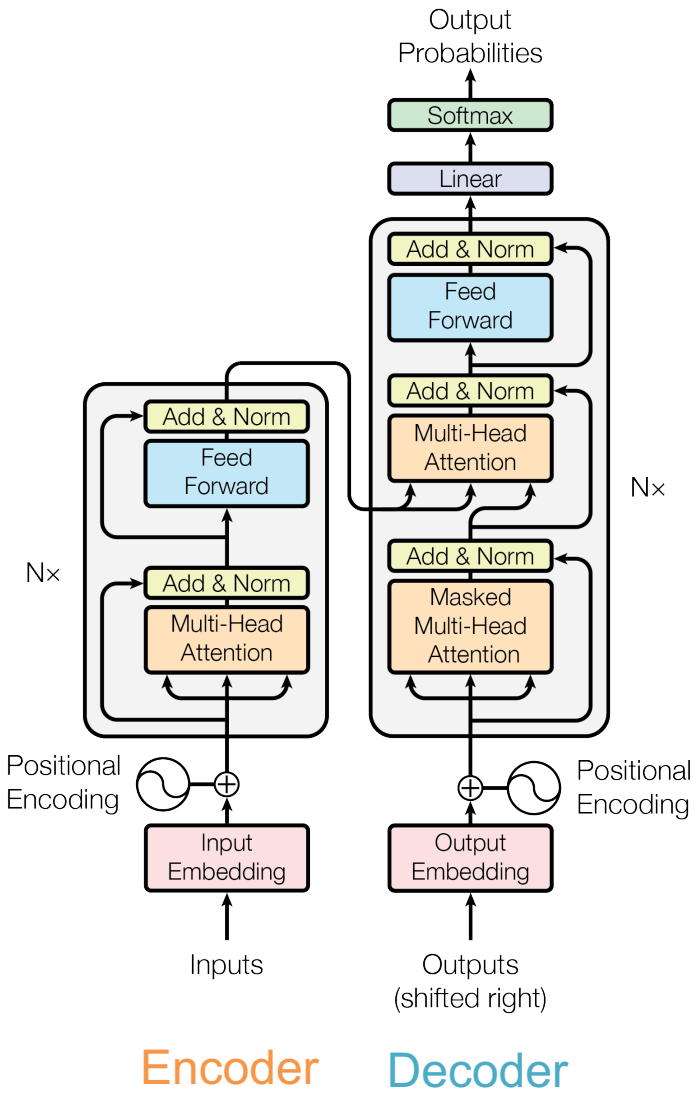
River bank

- ViT works the same as language transformer. The difference is in the preprocessing of the inputs.
  - Words ➔ ?

James Briggs “ViT explained + fine tuning in Python”

# Structure of a transformer

- Encoder maps input to useful represenation, learns represenations.
- Decoder decodes rep and combines it with other input to predict output.
  - Encoder only e.g BERT, useful for learning representations.
  - Decoder only: GPT-3, useful for generation tasks.
  - Encoder-decoder = seq2seq (translation).
- ViT = encoder only. Learn how to represent images for various purposes.
- Both encoder and decoder contain **attention blocks**.



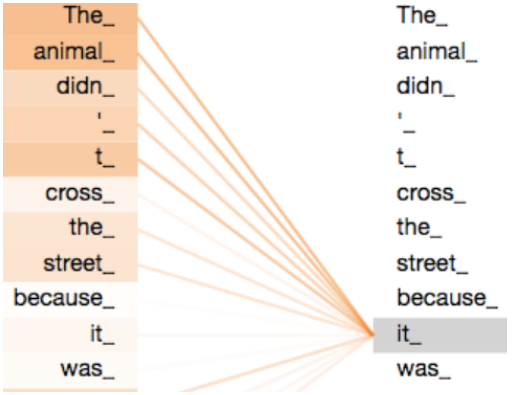

"Attention is all you need" , Google research 2017
Cited by > 151375

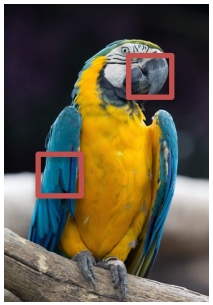
Source: Samuel Albanie https://www.youtube.com/watch?v=vsqKGZT8Qn8

# What is Attention?

Attention: a machine learning mechanism that allows a model to focus on the most relevant parts of the input when performing tasks.

- Examples:
  - **In Language Translation:** attention helps the model focus on the correct words from the input language that match the words it is generating in the output language.
  - **In Images**: to identify a parrot in an image, attention helps to focus on features of a parrot (beak, feathers) rather than on the background.

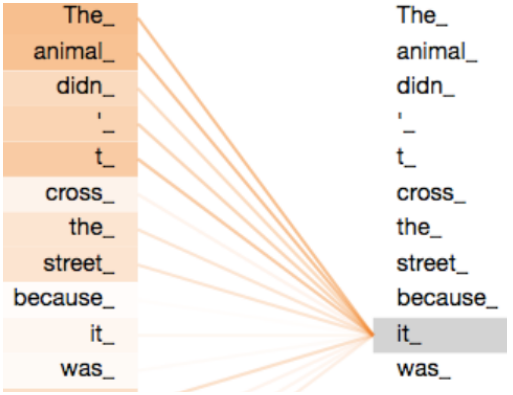


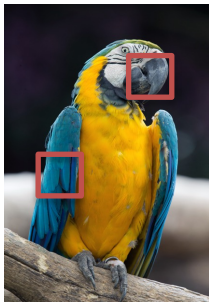



# What is Attention?

Attention: a machine learning mechanism that allows a model to focus on the most relevant parts of the input when performing tasks.

- Examples:
  - **In Language Translation:** attention helps the model focus on the correct words from the input language that match the words it is generating in the output language.
  - **In Images**: to identify a parrot in an image, attention helps to focus on features of a parrot (beak, feathers) rather than on the background.

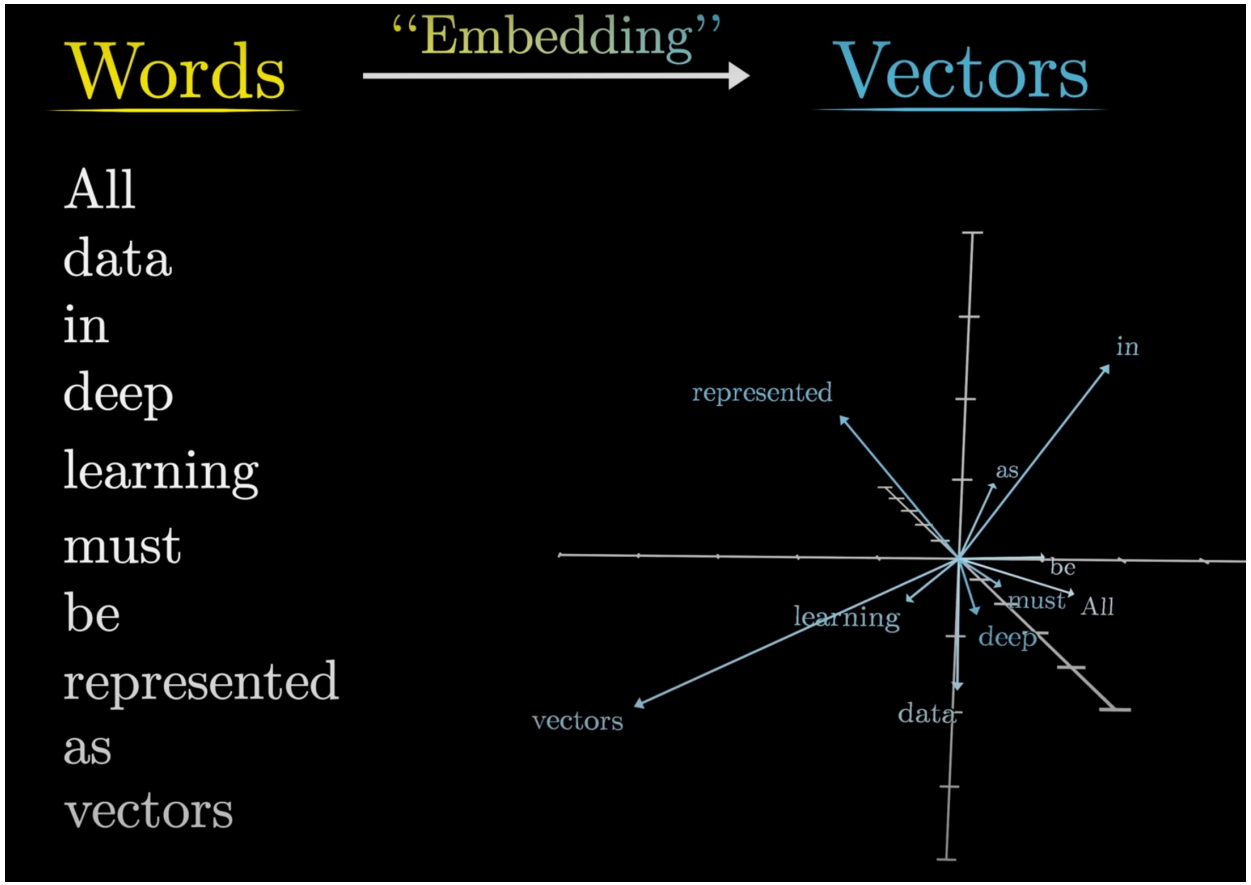


- How does it work?
  - Start with looking at **all parts** of the input.
  - Learn the network weights that distill **relevant relationships** between parts of the input
  - Learn multiple copies, each can focus on relevance to a **different task/ context**.

# Attention explained on a text example

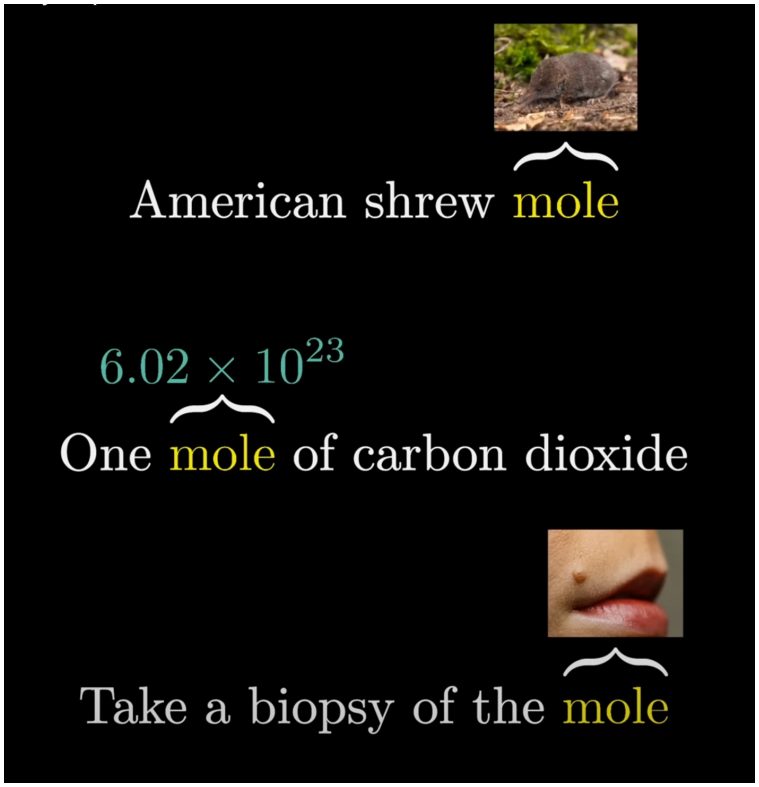
- A word is embedded as a vector in multidimensional space (many options here, some are well established).

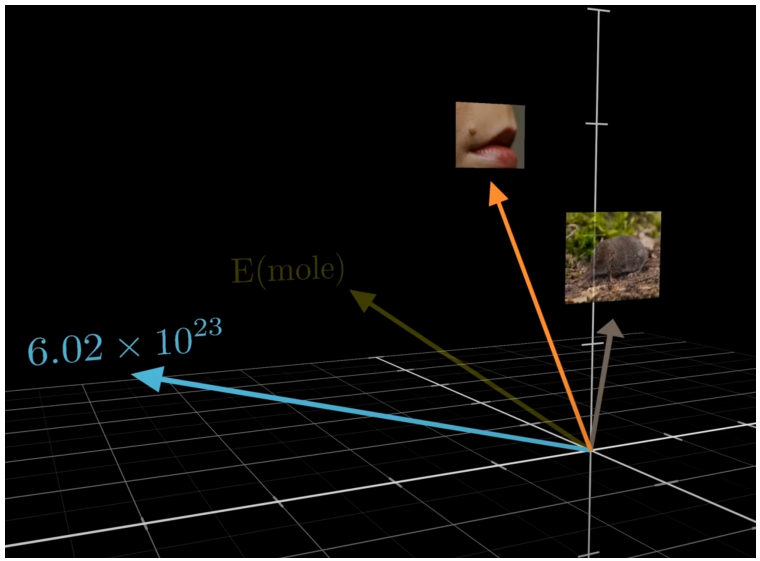



3Blue1Brown: "attention in transformers visually explained"
https://www.youtube.com/watch?v=eMlx5fFNoYc&t=559s

# Words are embedded as vectors

- A word is embedded as a vector in multidimensional space.
- Directions and sizes represent meanings.

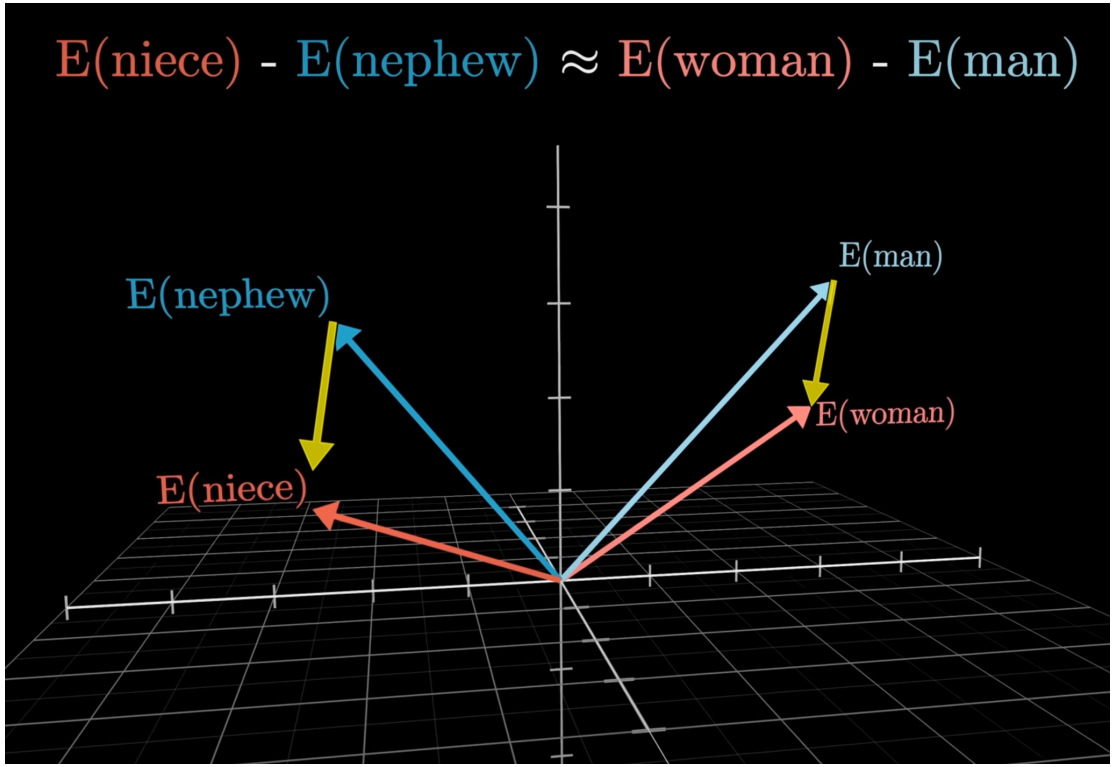



3Blue1Brown: “attention in transformers visually explained” https://www.youtube.com/watch?v=eMlx5fFNoYc&t=559s

# Transform vectors to update meanings

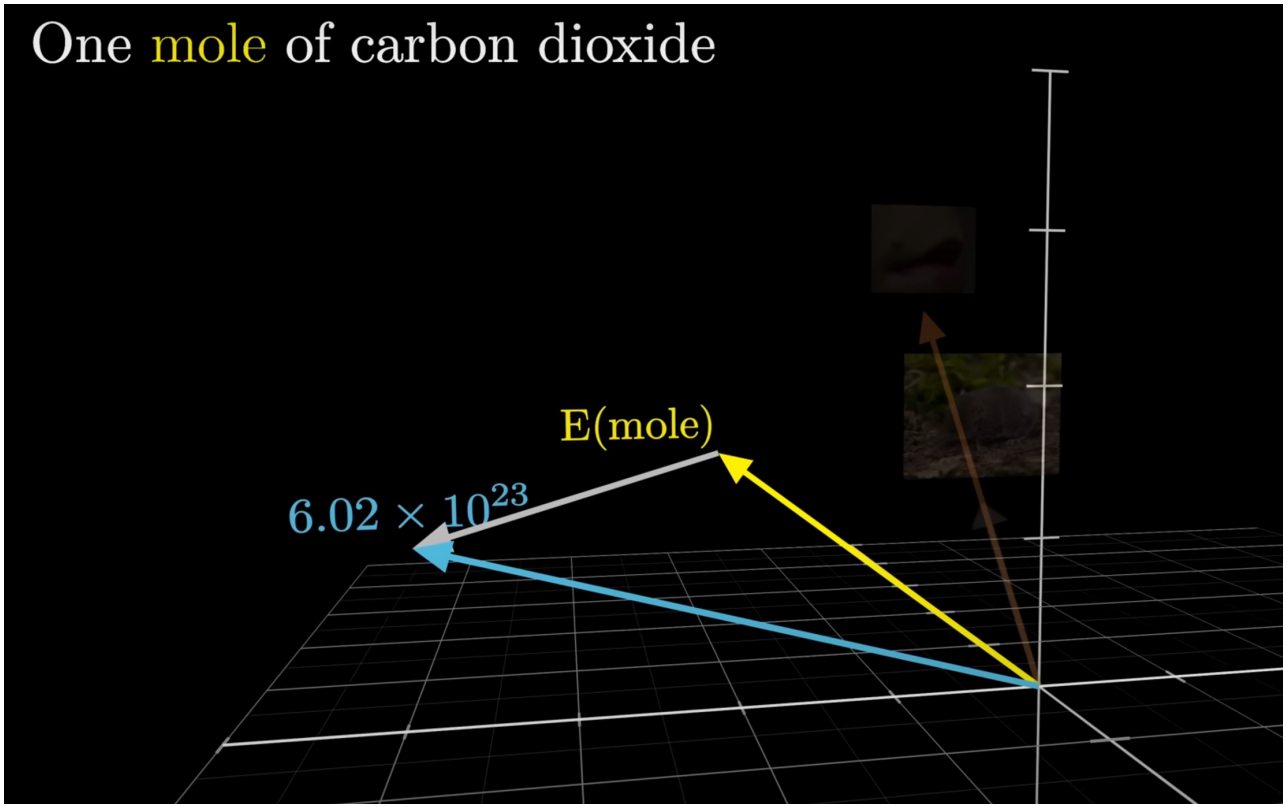
- A word is embedded as a vector in multidimensional space.
- Directions and sizes represent meanings.
- Shifting vectors = modifying meaning:




3Blue1Brown: “attention in transformers visually explained” https://www.youtube.com/watch?v=eMlx5fFNoYc&t=559s

# Attention is aimed at learning meaningful representations

- A well trained attention block learned to move the original vector to its refined direction corresponding to the right meaning.
- It learned it using the context.

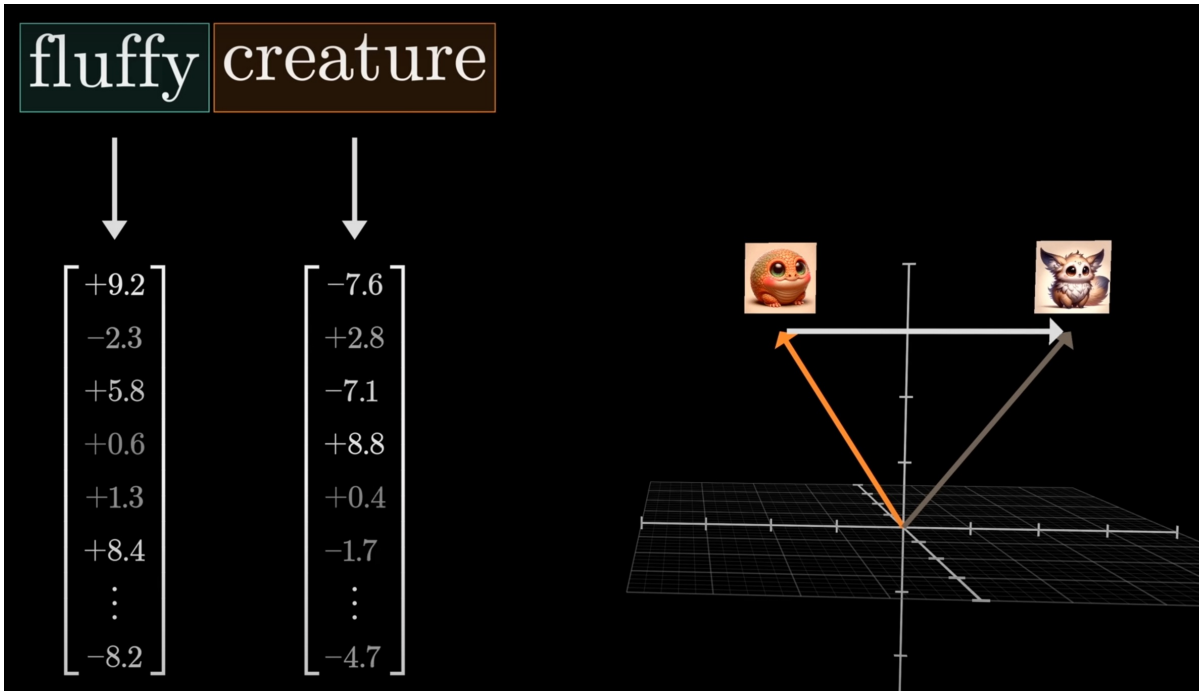



3Blue1Brown: “attention in transformers visually explained” https://www.youtube.com/watch?v=eMlx5fFNoYc&t=559s

# Attention is aimed at learning meaningful representations

- A well trained attention block learned to move the original vector to its refined direction corresponding to the right meaning.
- It learned it using the context.

Example: update “creature” to “fluffy creature”.




3Blue1Brown: “attention in transformers visually explained” https://www.youtube.com/watch?v=eMlx5fFNoYc&t=559s

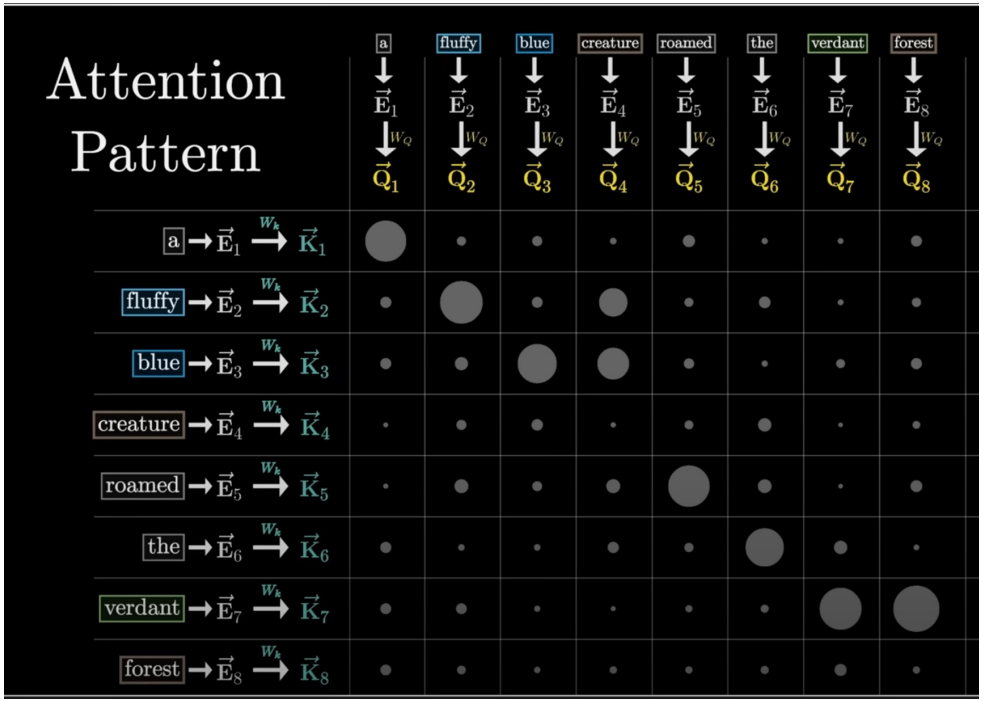
# Attention: tokens "listening" to each other

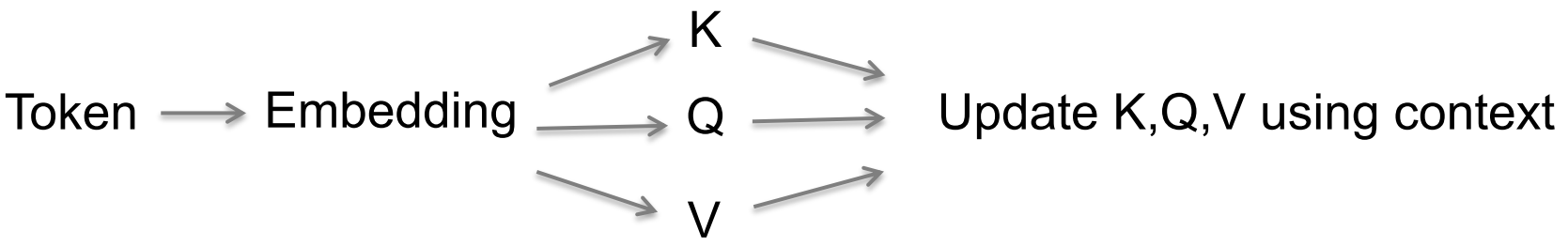
- Each word asks:
  - What info do I have? **K**ey vector
  - What do I need? **Q**uery vector
  - What do I communicate? **V**alue vector
- K,Q,V get updated to learn only relevant connections.
- Relevant is when $K_i$ is aligned with $Q_j$.


3Blue1Brown: "attention in transformers visually explained" https://www.youtube.com/watch?v=eMlx5fFNoYc&t=559s

# Attention: tokens "listening" to each other

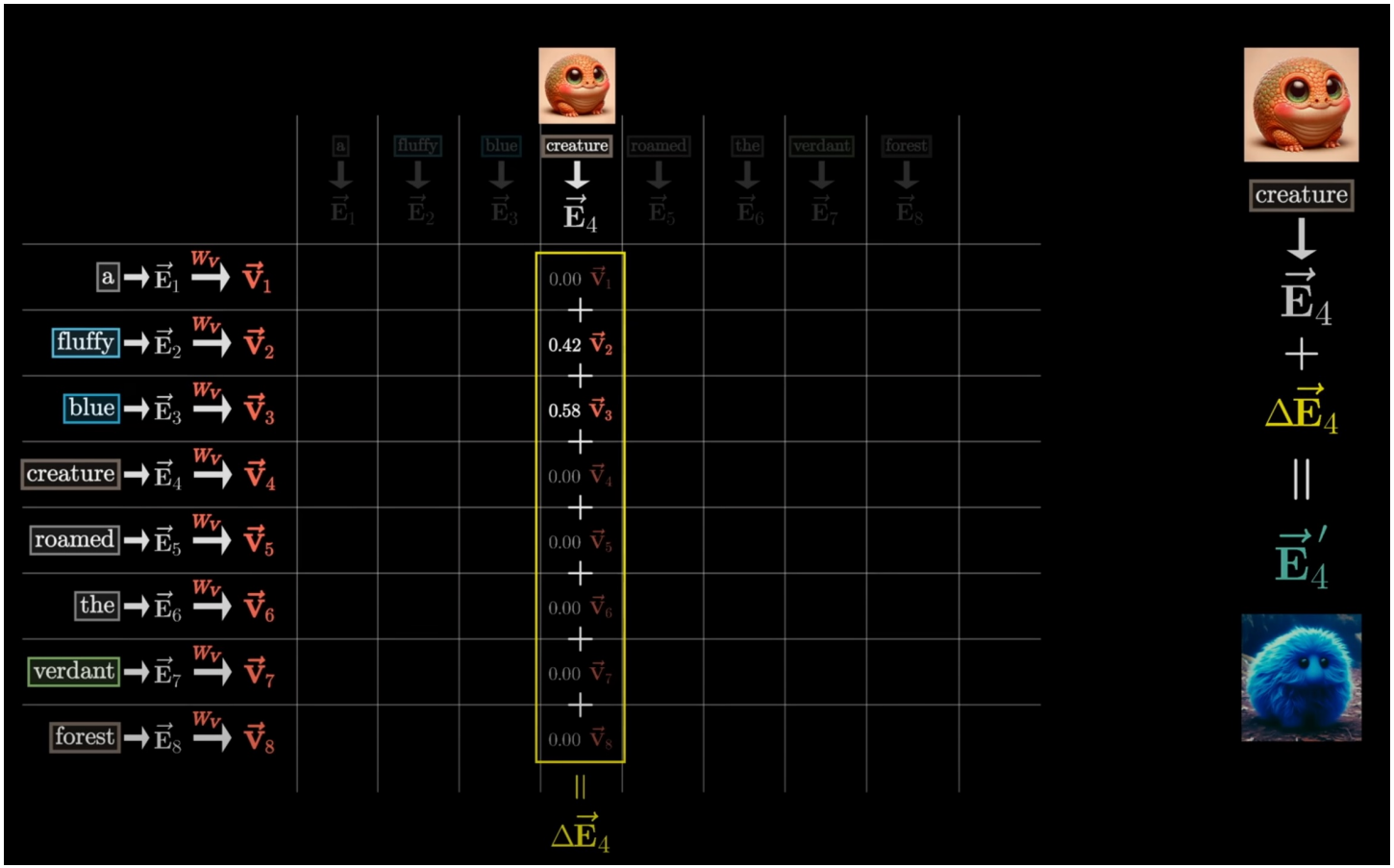
- Each word asks:
  - What info do I have? **K**ey vector
  - What do I need? **Q**uery vector
  - What do I communicate? **V**alue vector
- K,Q,V get updated to learn only relevant connections.
- Relevant is when $K_i$ is aligned with $Q_j$.




3Blue1Brown: "attention in transformers visually explained" https://www.youtube.com/watch?v=eMlx5fFNoYc&t=559s

# Attention: tokens refining each other's embedding

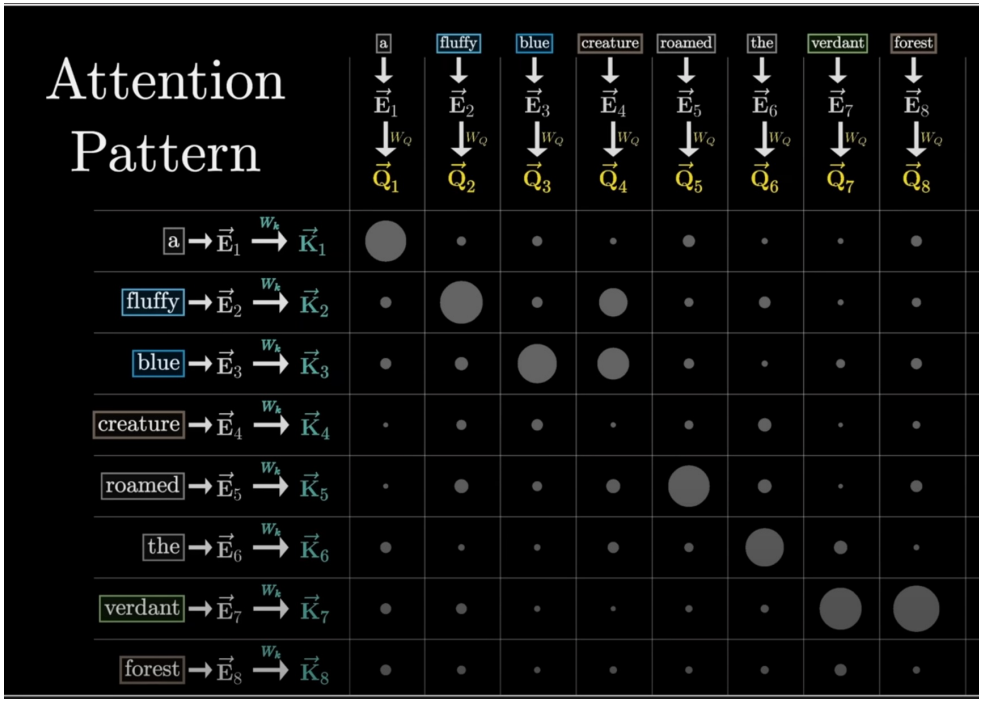



- "Creature" is refined to "fluffy creature" because "fluffy" updates the embedding of "creature".
- Update: multiply the "creature" vector by a value matrix and add to "creature".
- The value matrix: a weighted average of all V vectors.

# Attention: the math of transformed vectors

Learning process:

- Adjust Q,K,V to refine meanings using the other words.



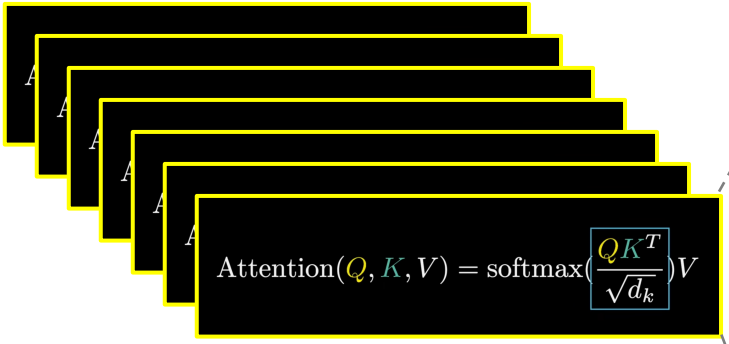

Summarize the output of one attention head:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

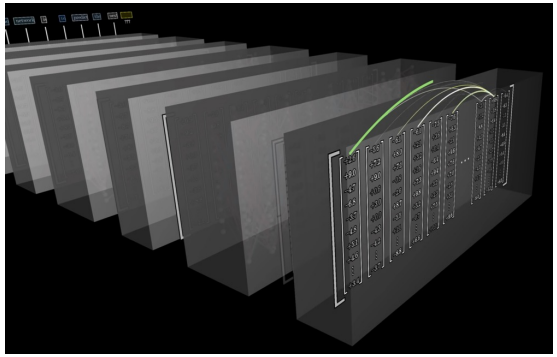
3Blue1Brown: “attention in transformers visually explained” https://www.youtube.com/watch?v=eMlx5fFNoYc&t=559s

# Attention is all you need?

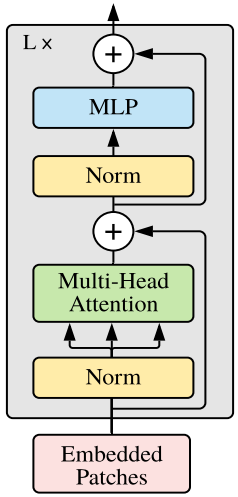
- You need attention. Trained to update embeddings.




- You need **multi-head** attention. Heads trained in parallel, each embeds a different context.



- You need **depth**. Deeper = broader context, sentiments, tone...



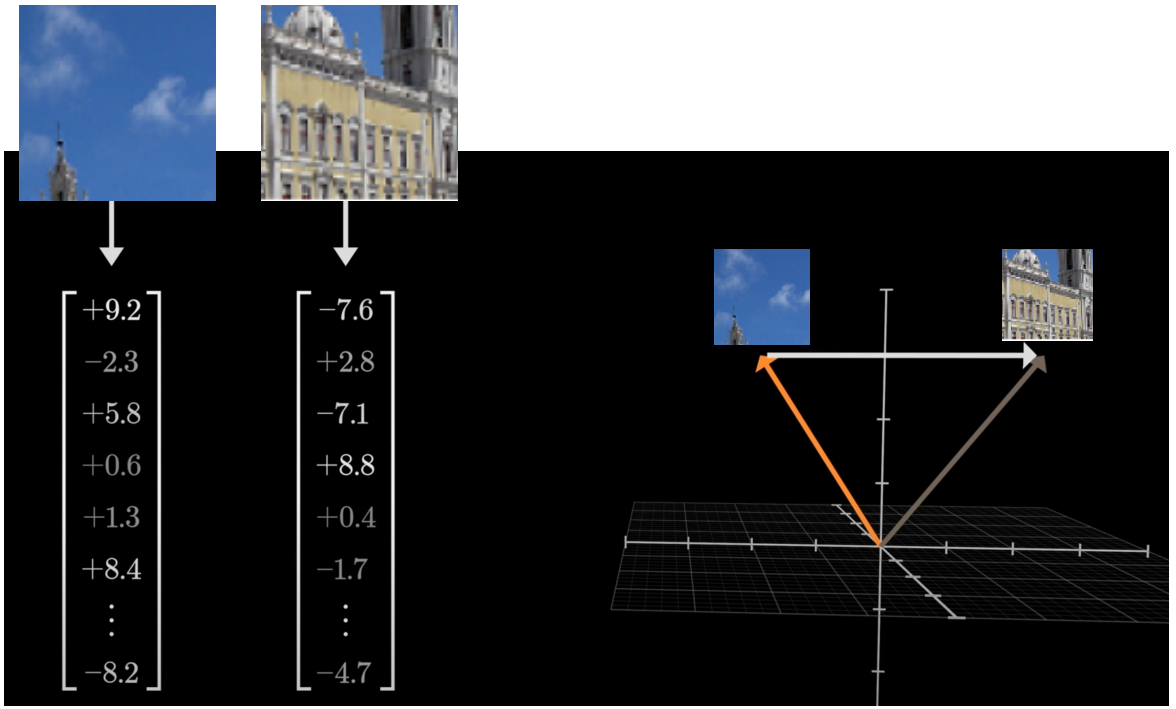

- You need quite some more **tricks**. Let us dive into the details of the vision transformer.

# Transformers for Image Data

AN IMAGE IS WORTH 16X16 WORDS:
TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE  Dosovitskiy et al. (Google) 2020.

**Step 1: Split the image to patches.**

- Sentence = list of tokens (words).
- Image = a list of patches.




Clear explanation: James Briggs “ViT explained + fine tuning in Python”

# Transformers for Image Data

AN IMAGE IS WORTH 16X16 WORDS:
TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE Dosovitskiy et al. (Google) 2020.

**Step 1: Split the image to patches.**

- Sentence = list of tokens (words).
- Image = a list of patches.

Why patches and not pixels?

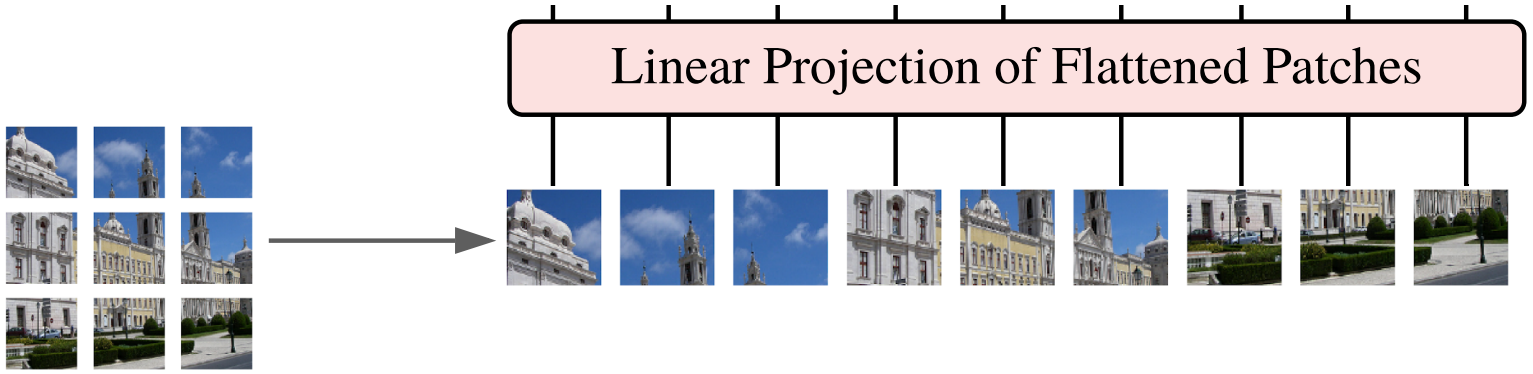
- Image of 224 x 224 pixels ➔ Mutual attention requires $224^4$= 2.5 billion comparisons. Just for a single attention layer, and we need many.
- 14 x 14 patches ➔ 40'000 comparisons.

Clear explanation: James Briggs "ViT explained + fine tuning in Python"

# Transformers for Image Data

**Step 2: Embed each patch using linear projection.**

- This reformats each patch as a vector.

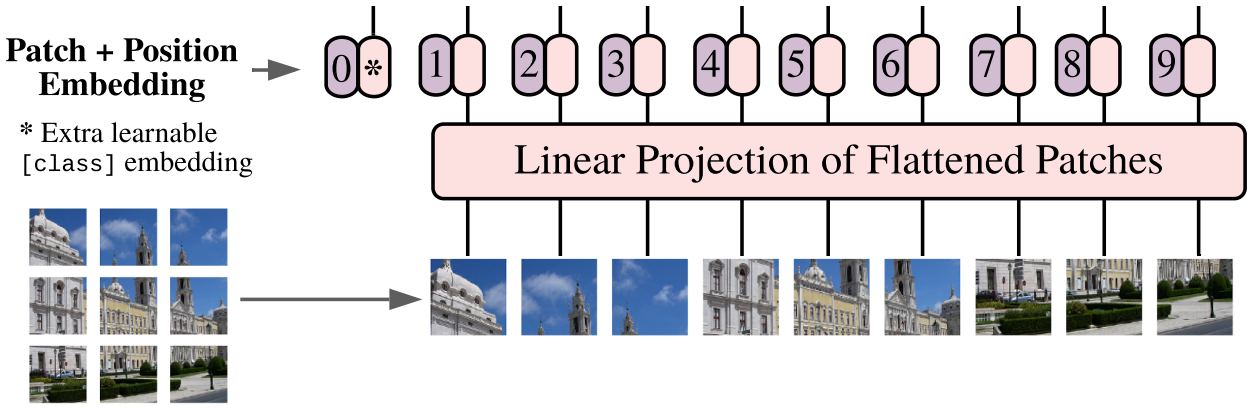



Clear explanation: James Briggs “ViT explained + fine tuning in Python”

# Transformers for Image Data

**Step 3: learnable class embedding = class token**.

- Same as CLS token in BERT.
- An embedding of the whole sentence/image in a single token.
- Will be passed to the classification head.
- Even more important for ViT, where one of the main tasks is classification.




Clear explanation: James Briggs “ViT explained + fine tuning in Python”

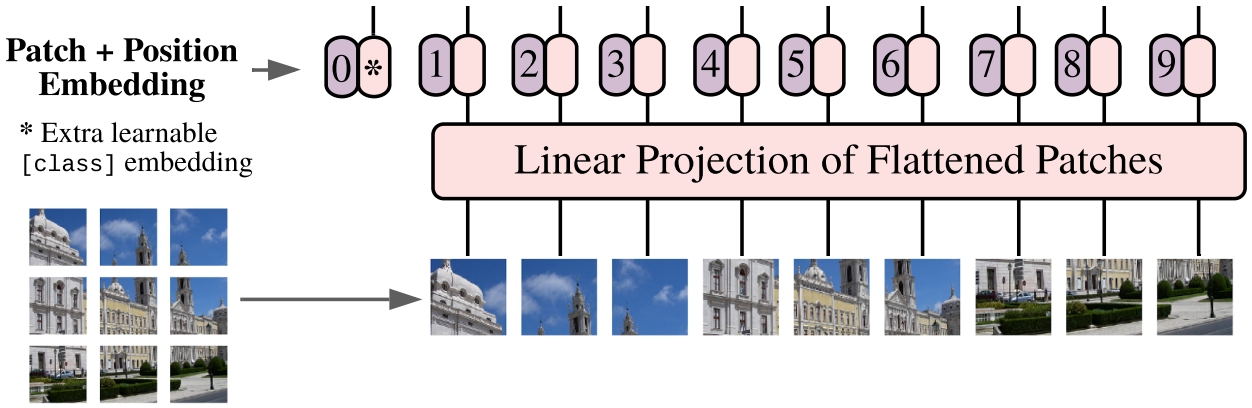
# Transformers for Image Data

**Step 3: learnable class embedding = class token**.

- Same as CLS token in BERT.
- An embedding of the whole sentence/image in a single token.
- Will be passed to the classification head.
- Even more important for ViT, where one of the main tasks is classification.

**Step 4: positional embedding.**

- Order of words/patches makes a difference.
- Patches in a similar area are pushed together.
- Introduce the sense of locality into the embedding of the image.

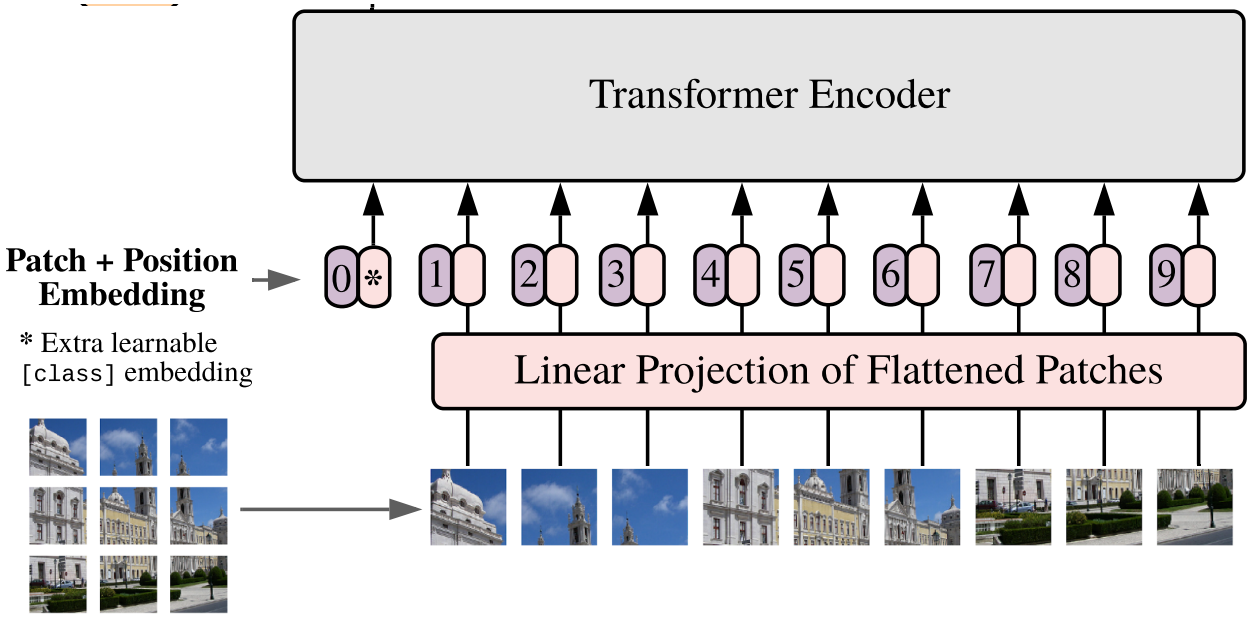



Clear explanation: James Briggs “ViT explained + fine tuning in Python”

# Transformers for Image Data

**Step 5: feed into a transformer encoder**.

- Similar to text transformer.
- 4 key ideas inside (see in a bit).

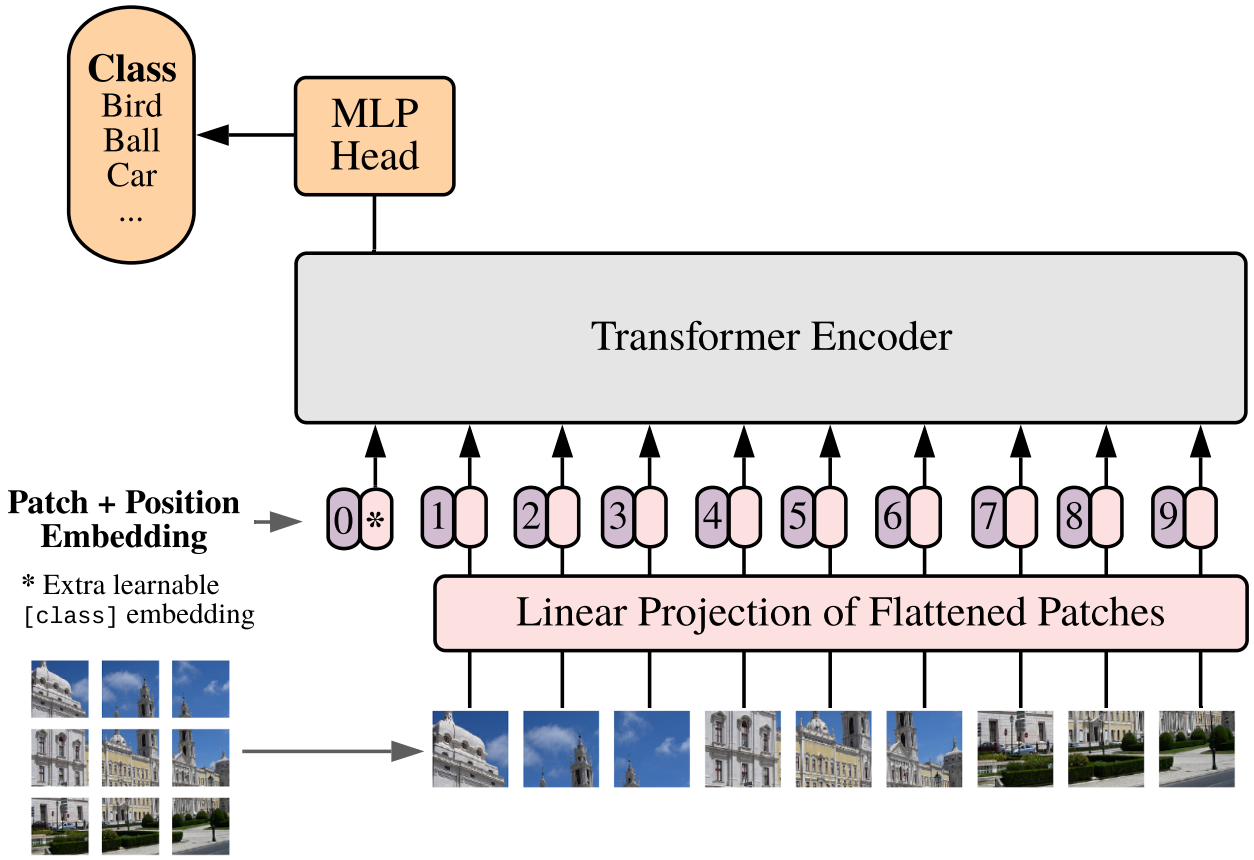



Clear explanation: James Briggs “ViT explained + fine tuning in Python”

# Transformers for Image Data

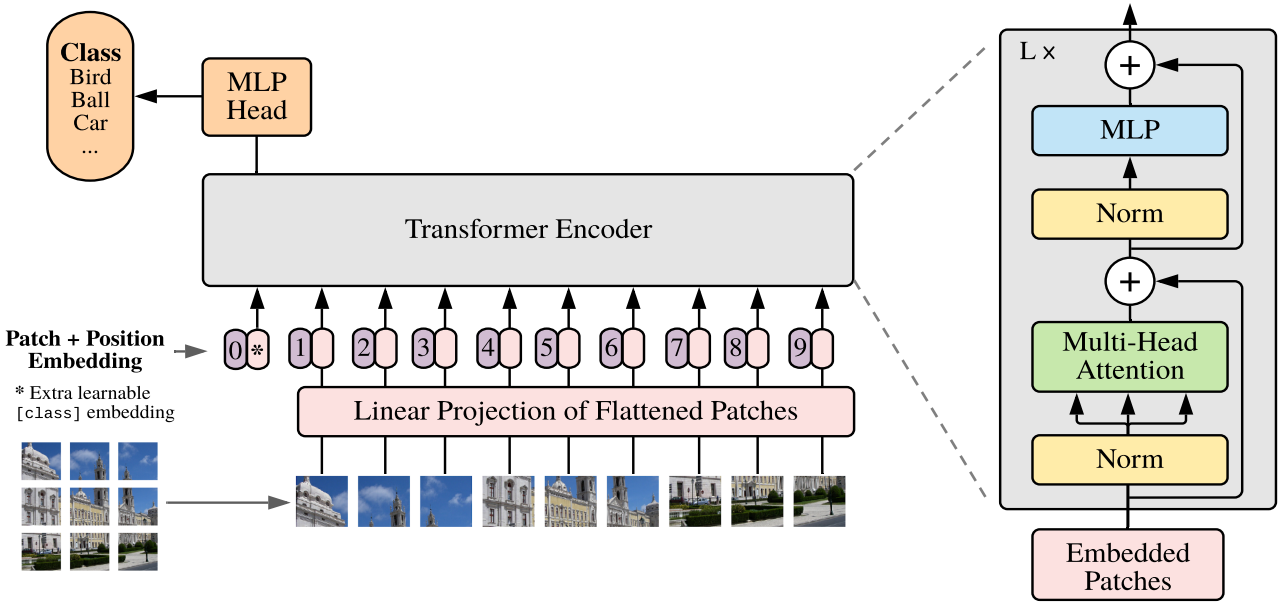
**Step 6: pass to MLP head.**

**Step 7: predict class.**




Clear explanation: James Briggs “ViT explained + fine tuning in Python”

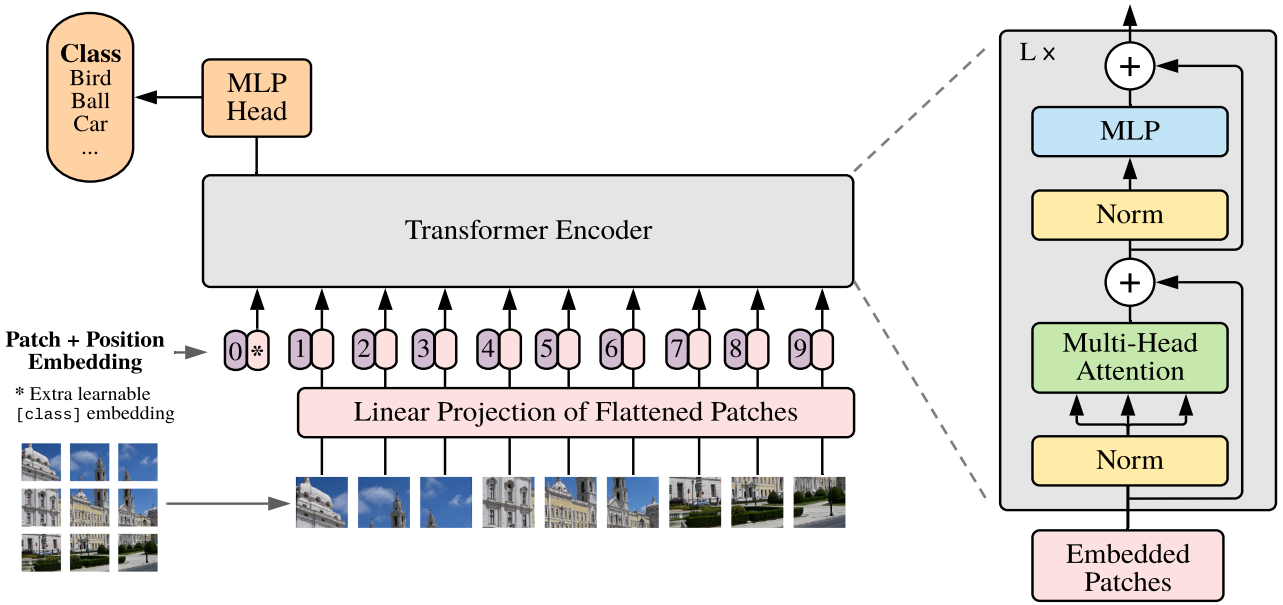
# Structure of the transformer encoder




The encoder includes a sequence of L attention blocks

Clear explanation: Samuel Albanie https://www.youtube.com/watch?v=vsqKGZT8Qn8

# The attention block




Each attention block includes 4 key ideas

Allows embeddings to «think» independently

Layer normalization: stabilize learning

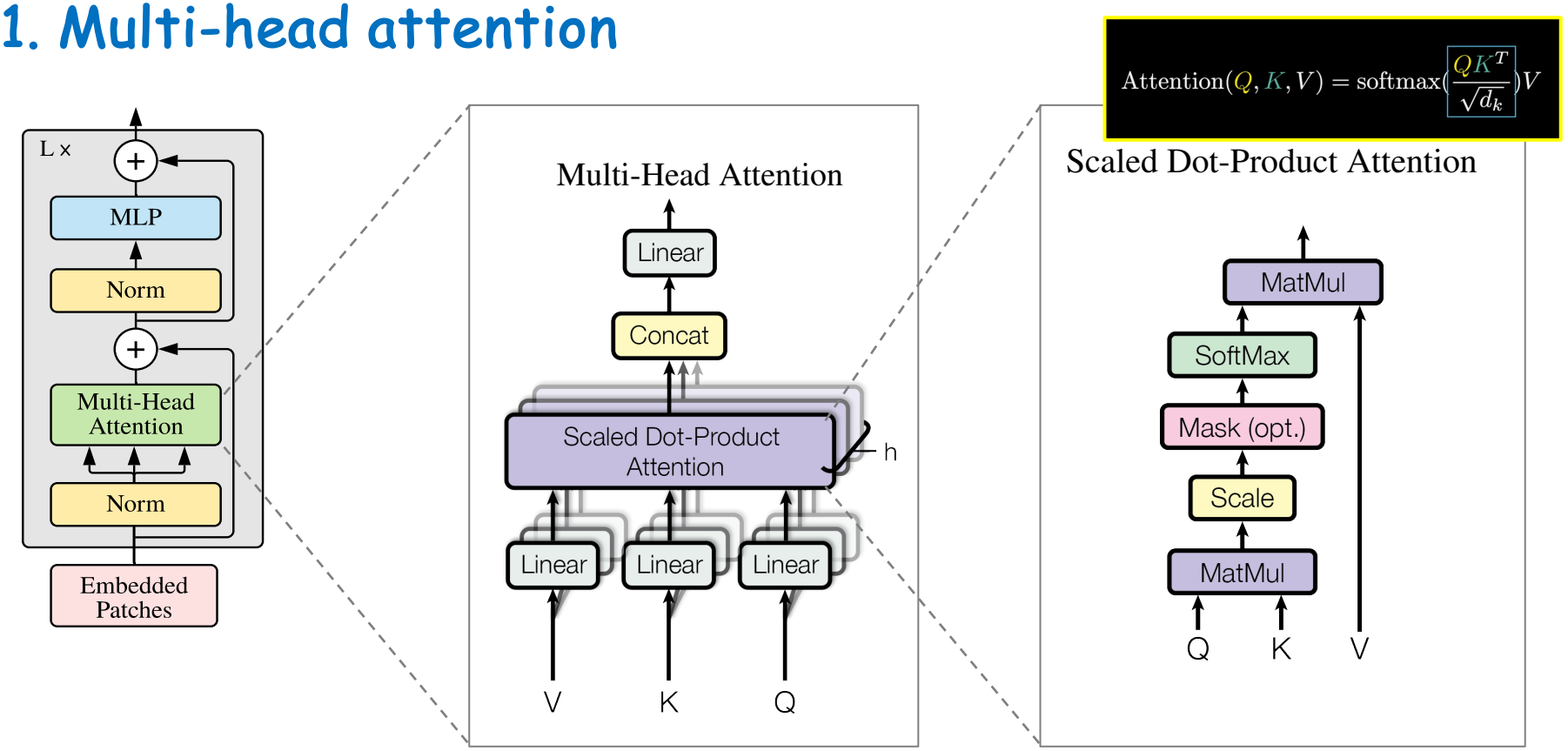
Allows embeddings to communicate

Residual connections: help gradient flow

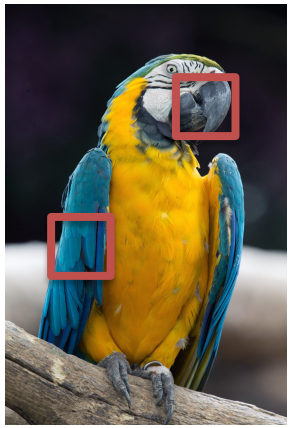
➔ Next: walk through the 4 key ideas

Clear explanation: Samuel Albanie https://www.youtube.com/watch?v=vsqKGZT8Qn8

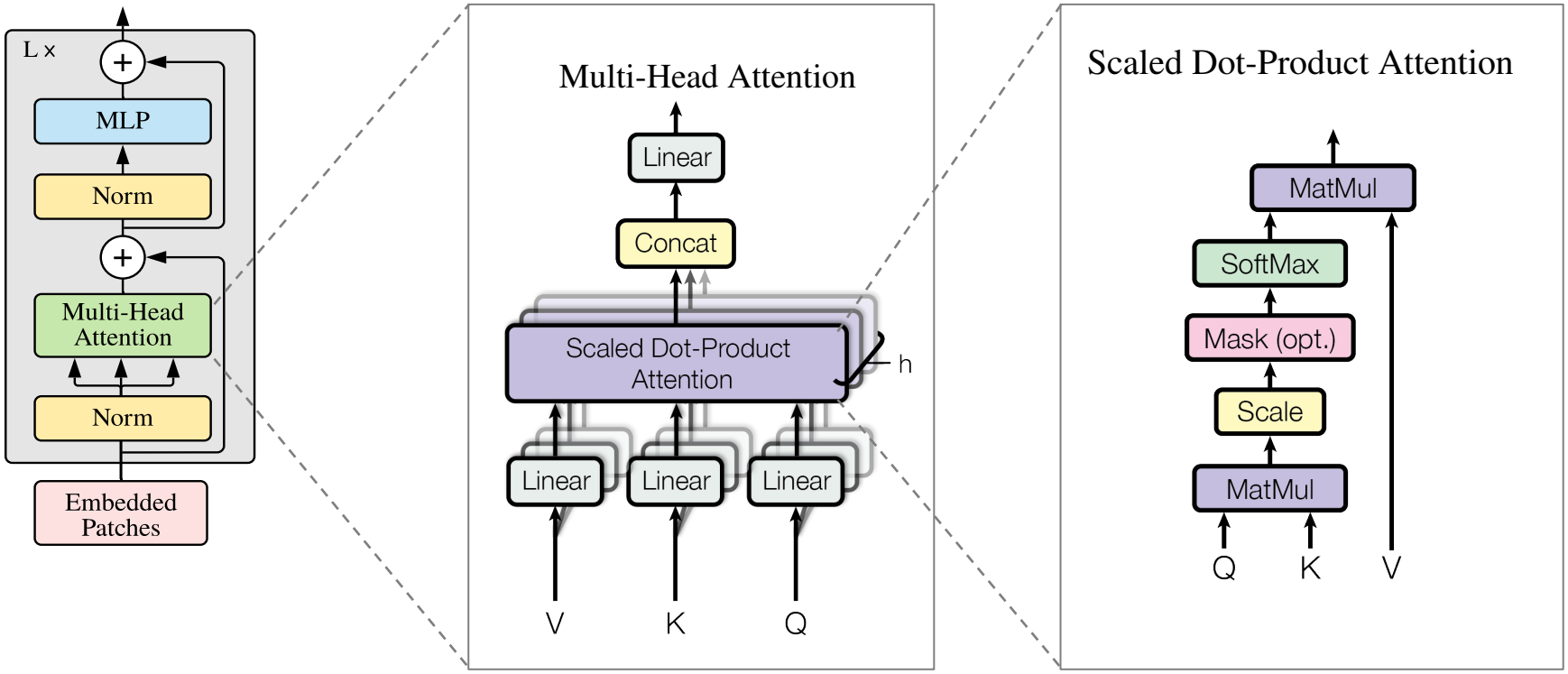
# 1. Multi-head attention




- Each patch asks:
  - What info do I have? **K**ey vector
  - What do I need? **Q**uery vector
  - What do I communicate? **V**alue vector
- K, Q, V get updated to learn only relevant connections.
- Relevant is when $K_i$ is aligned with $Q_j$.
- Beak + feathers ➔ parrot

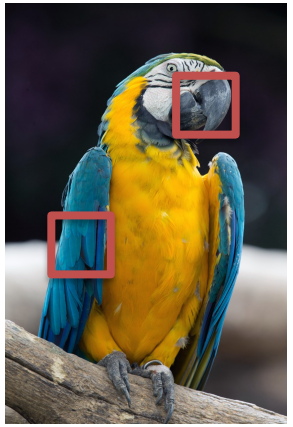


# 1. Multi-head attention

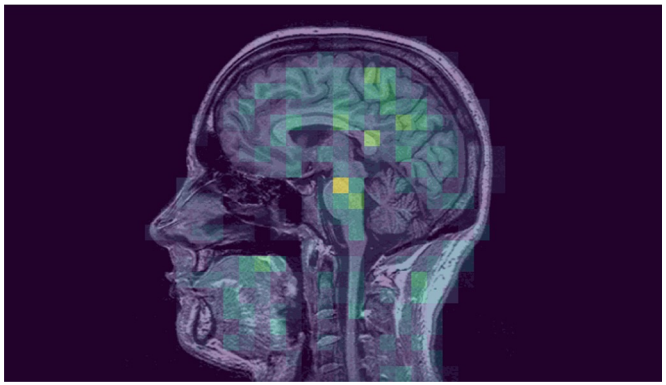


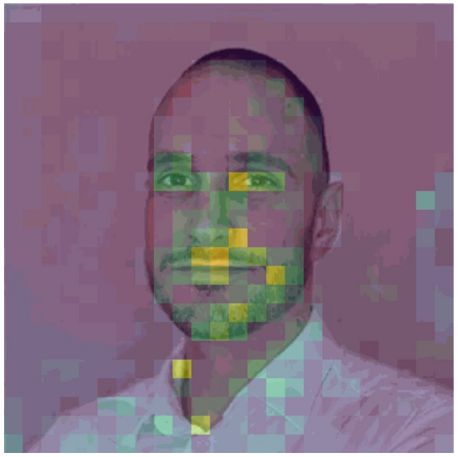

- h attention heads, each with its own Q,K, and V matrices.
- Different heads focus on different contexts/ tasks.
- All heads trained in parallel.
- Then concatinate outputs of all heads.

- Operations are well optimized on GPUs.
- Complexity scales as $N^2$, N is sequence length – expensive!

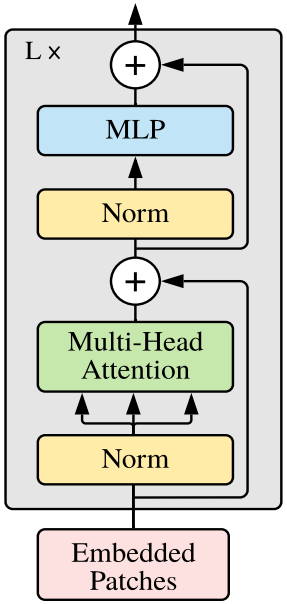


# 1. Multi-head attention





# 2. MLP




**Attention = "Where should I look?"** mixes token information globally.

**MLP = "What should I do with what I saw?"** processes each token separately to extract richer features:

$$MLP(\vec{E}) = W_2 \cdot GELU(W_1 \cdot \vec{E} + b_1) + b_2$$

Embedding of a single patch

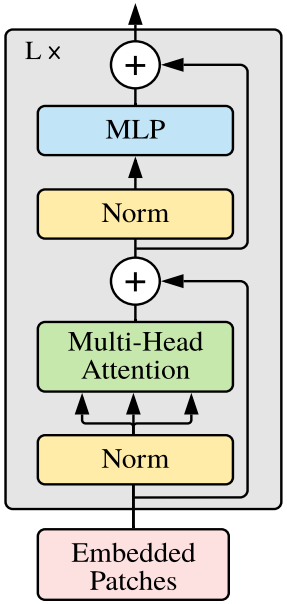
non-linearity

Usually 2 fully connected layers

➔ Transform the feature space in a way that allows the network to model more complex functions.

Source: Samuel Albanie https://www.youtube.com/watch?v=vsqKGZT8Qn8

# 3. Layer normalization




- Prepare each patch embedding so the attention mechanism and the MLP can work on a stable, normalized feature space.

➔ Training is stable even with very deep stacks of transformer blocks.

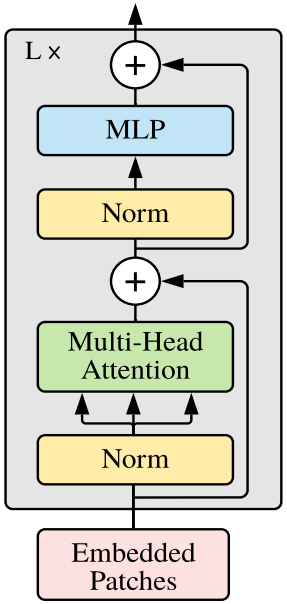
Source: Samuel Albanie https://www.youtube.com/watch?v=vsqKGZT8Qn8

# 4. Residual connections

$$x_{attn} = x + \text{MultiHeadSelfAttention}(\text{LayerNorm}(x))$$

mix information across tokens

for stability




Why add the input back?

1. Preserve original patch identity.
2. Stability across layers.
3. Better gradient flow.

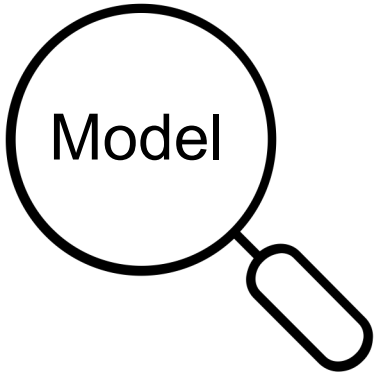
Source: Samuel Albanie https://www.youtube.com/watch?v=vsqKGZT8Qn8

# Why understand attention?

- Models are too large to train on your own
  - tiny ViT ~6M, huge ViT ~600M trainable parameters
- Common use: download (e.g from hugging face) and infere or fine tune.

- But: we can still improve performance by understanding what the model sees and learns.
  - Modify your data (preprocess, augment, clean…)
  - Select another pretrained ViT (larger, different training data…)
  - Try fine tuning

➔ Learn how to investigate your



# Learned position relations

In ViT: no inherent spatial structure like in CNNs ➔ rely on **learned** position embeddings to inject spatial information.

**1. Extract the learned position embeddings**

```
pos_embed = model.pos_embed  # shape: (1, 7x7+1, D=1024)
pos_embed_patches = pos_embed[0, 1:, :]  # shape: (49, 1024)
```

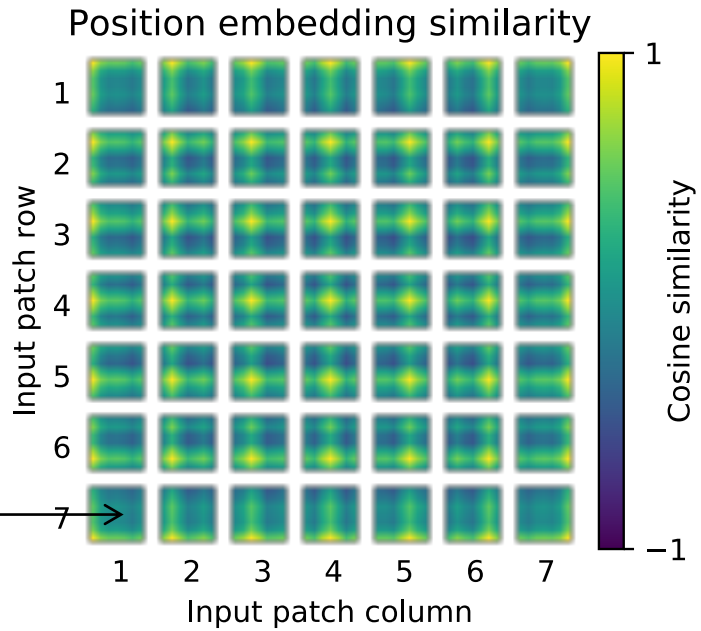
**2. Select one patch location (i, j)**

```
anchor = pos_grid[i, j] #shape:(1024,)
```

**3. Compute cosine similarity with all others**

```
similarity_map =
cosine_similarity(pos_grid, anchor[None,
None, :], dim=-1) # shape: (7, 7)
```

similarity between this position and position embeddings of all other patches.



# Learned input embeddings

1. **Extract the patch embedding matrix E:** linearly projects the patches

P x P x C x D = 3072 x 1024

32 patch dim | 3 colors | 1024 embedding neurons

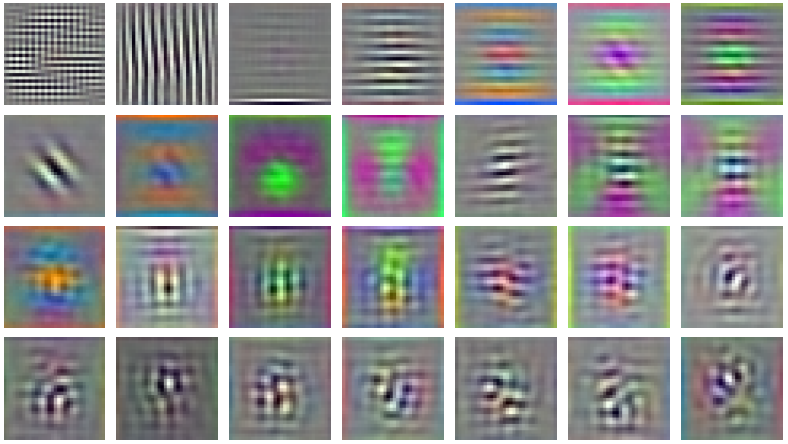
$$E \in \mathbb{R}^{D \times (P^2 \cdot C)} = \mathbb{R}^{1024 \times 3072}$$

```
E = model.patch_embed.proj.weight #of a pretrained ViT
```

2. **PCA on the columns of $E^T$ ➔** reduce from 1024 to 28 components

3. **Reshape each component to P x P x C**

RGB embedding filters
(first 28 principal components)



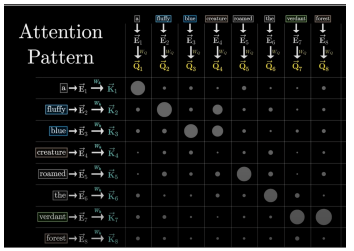
➔ top principal components of the patch embedding matrix E:
look like smooth spatial filters
e.g., edge detectors, gradients, etc.

➔ Even before attention layers, ViT learns structured patch-level filters.

# Attention distance

**1. Define patch positions** $\text{pos}(i) = (x_i, y_i) \quad \text{for } i = 1, \ldots, N$

**2. Calculate distance** $d_{ij} = \|\text{pos}(i) - \text{pos}(j)\|_2$




**3. Attention matrix per head** $A_{ij}$ = how much patch i attends to patch j

$$A_{\text{head}} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)$$

**4. Average over patches** $\bar{d}_i = \sum_{j=1}^{N} A_{ij} \cdot d_{ij}$

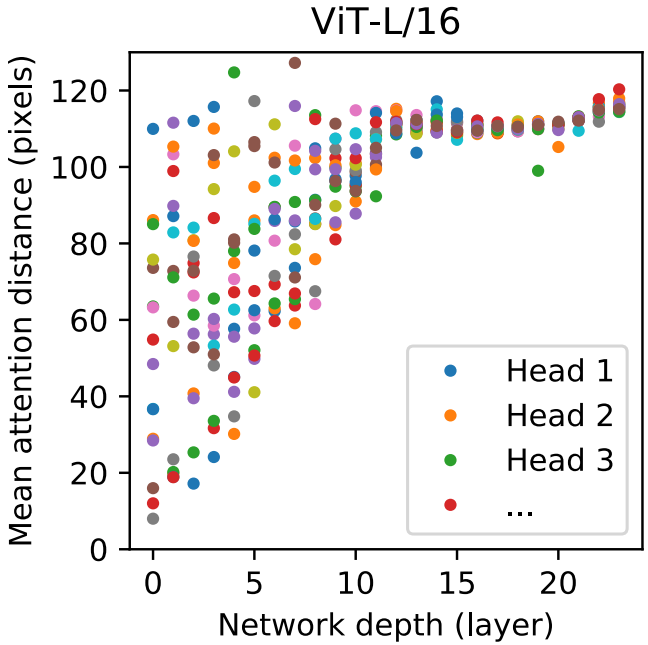
**5. Average over patches again**

$$\text{AvgDist} = \frac{1}{N}\sum_{i=1}^{N} \bar{d}_i$$

Conclude:

- Deep layers have global attention.
- Shallow layers have both local and global attention, depending on the head.

➔ Analogous to receptive field in CNNs

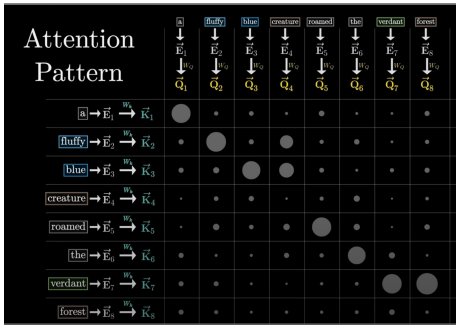


# Where does the transformer focus attention?

**1. Extract raw attention weights**
each attention head computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

➔ N×N matrix showing how each of the N tokens attends to the others in that head:



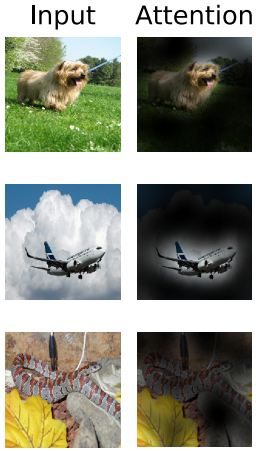

$$A_{\text{head}} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)$$

**2. Average across all Heads:**

$$A_{\text{layer}} = \frac{1}{h}\sum_{i=1}^{h} A^{(i)}$$ ← Attention matrix of head i

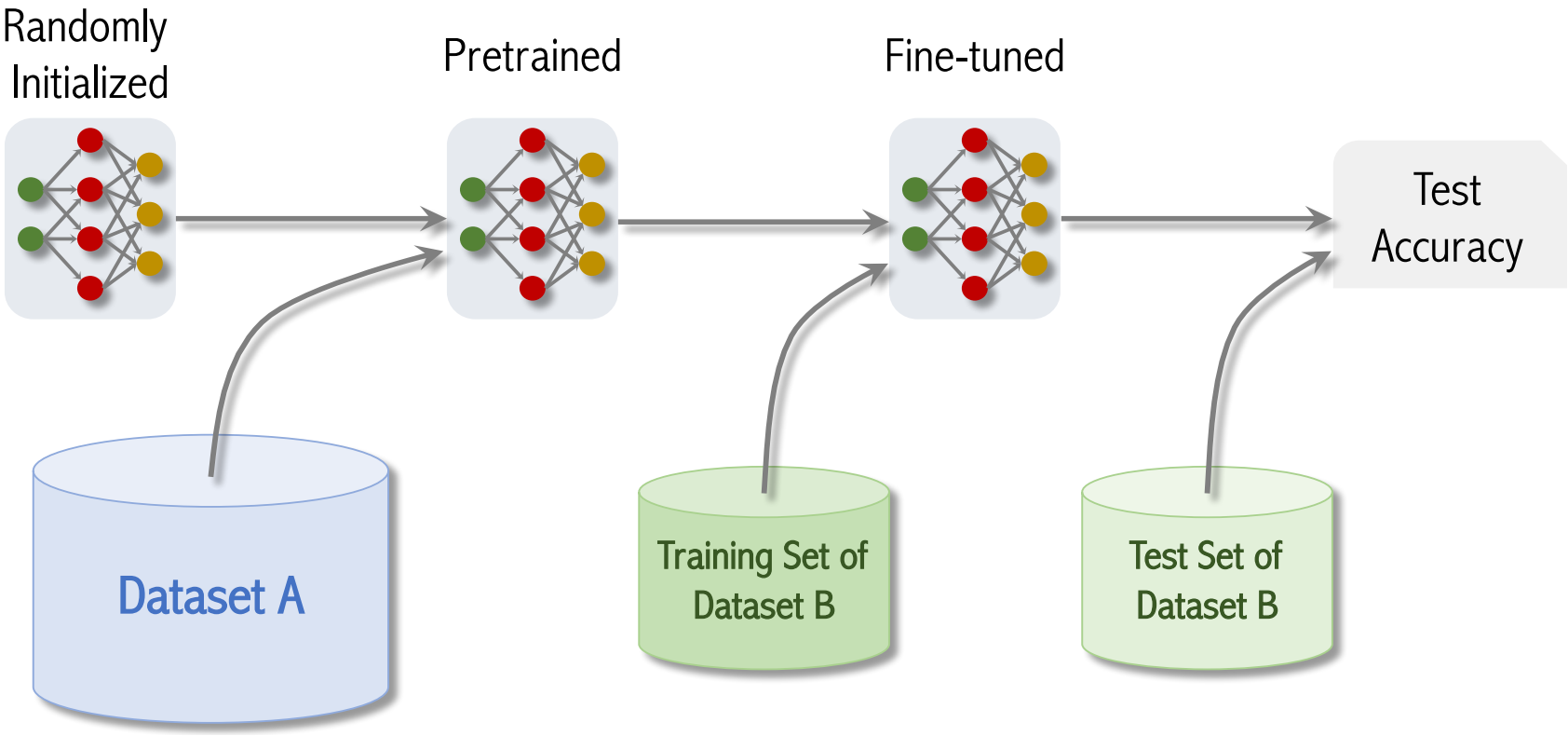
**3. Rollout across layers:**

$$A_{\text{rollout}} = A'^{(1)} \cdot A'^{(2)} \cdot \ldots \cdot A'^{(L)}$$



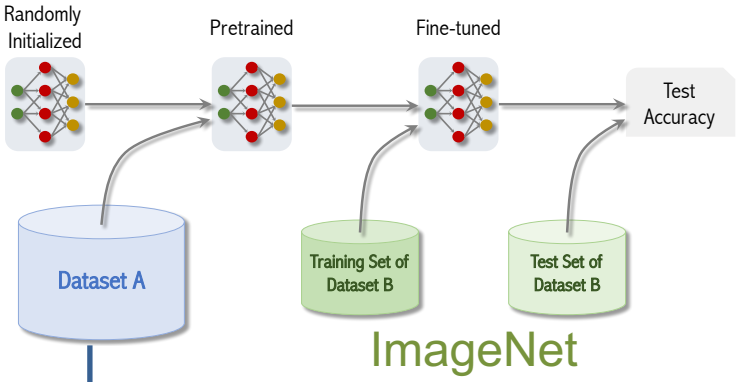

← first row = the attention from the class token to all other tokens (patches)
➔ patch-level attention

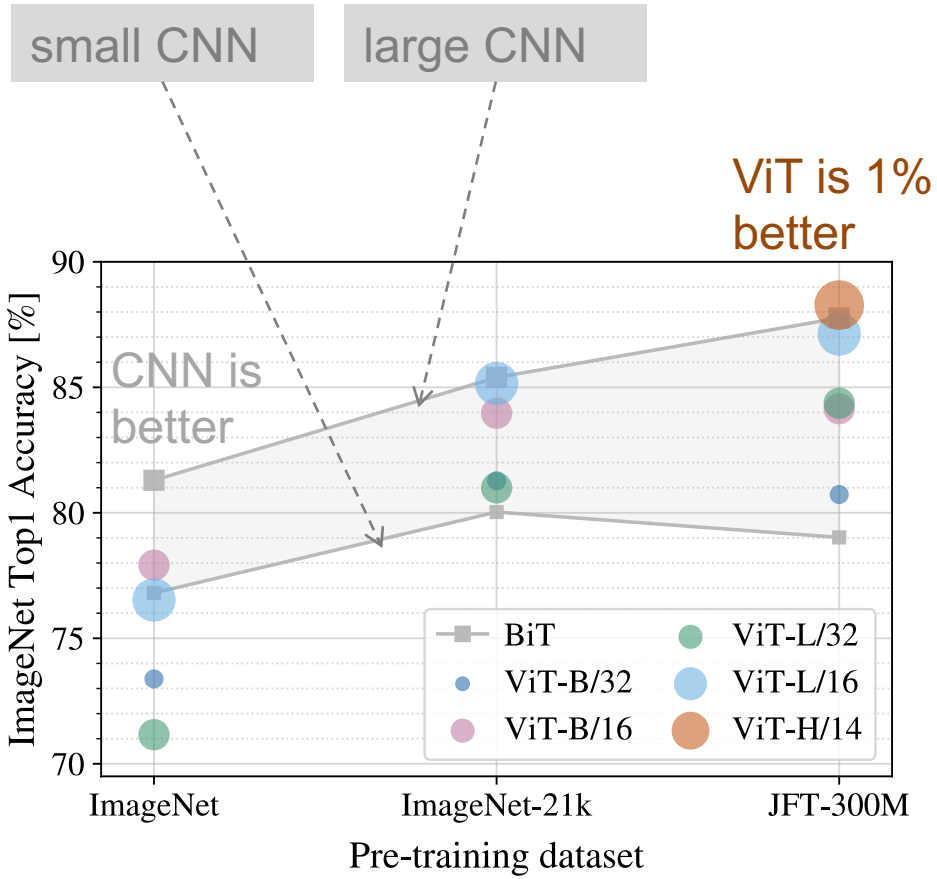
# Testing performance in a transfer classification task




Source: Shusen Wang https://github.com/wangshusen/DeepLearning/tree/master/Slides

# Testing performance in a transfer classification task

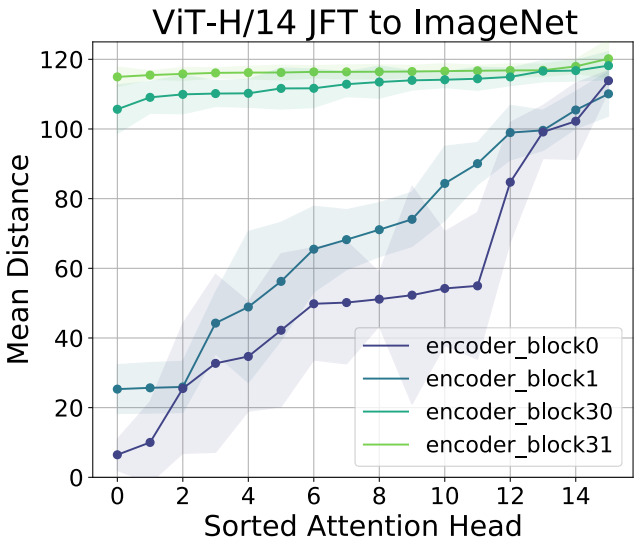



| Pretraining dataset A | ImageNet (no transfer) | ImageNet-21k | JFT |
|---|---|---|---|
| # images | 1.3 M | 14 M | 300 M |
| # classes | 1 K | 21 K | 18 K |

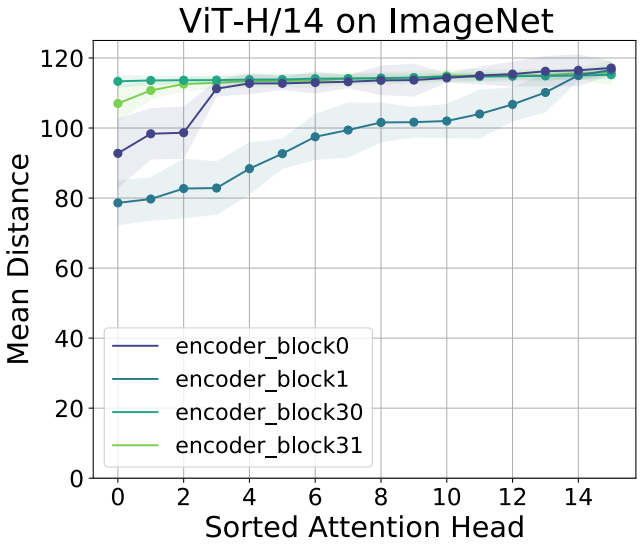
# Dataset size and attention distance

More training data




- Performs similarly to CNNs
- Lower attention layers learn to attend locally

Less training data

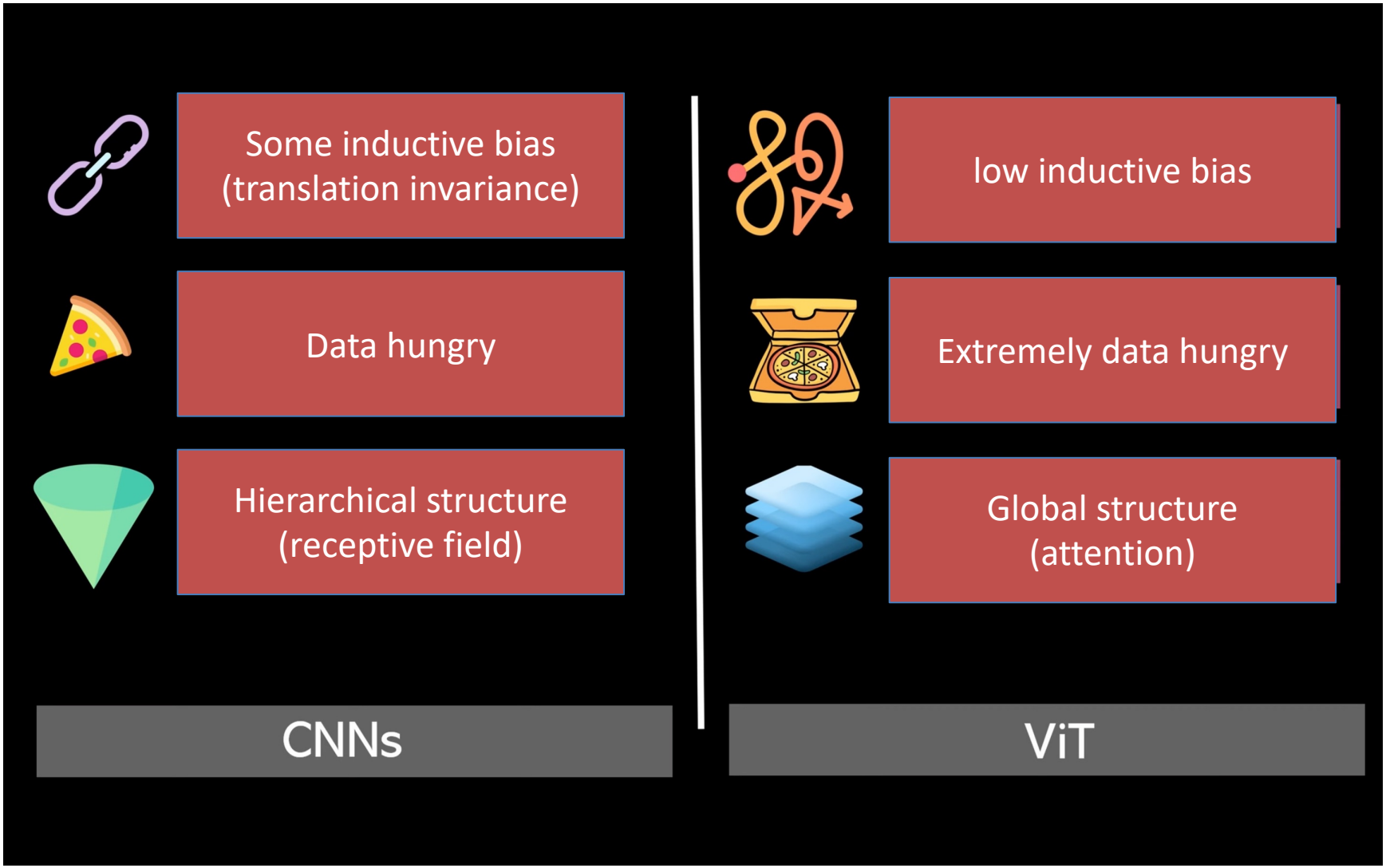



- Worse than CNNs
- All layers attend globally

Incorporating local features (hardcoded into CNNs) may be important for strong performance.

Raghu, Maithra, et al. "Do vision transformers see like convolutional neural networks?." (2021).

# ViT vs. CNN




Source: Vision Transformer Quick Guide https://www.youtube.com/watch?v=j3VNqtJUoz0

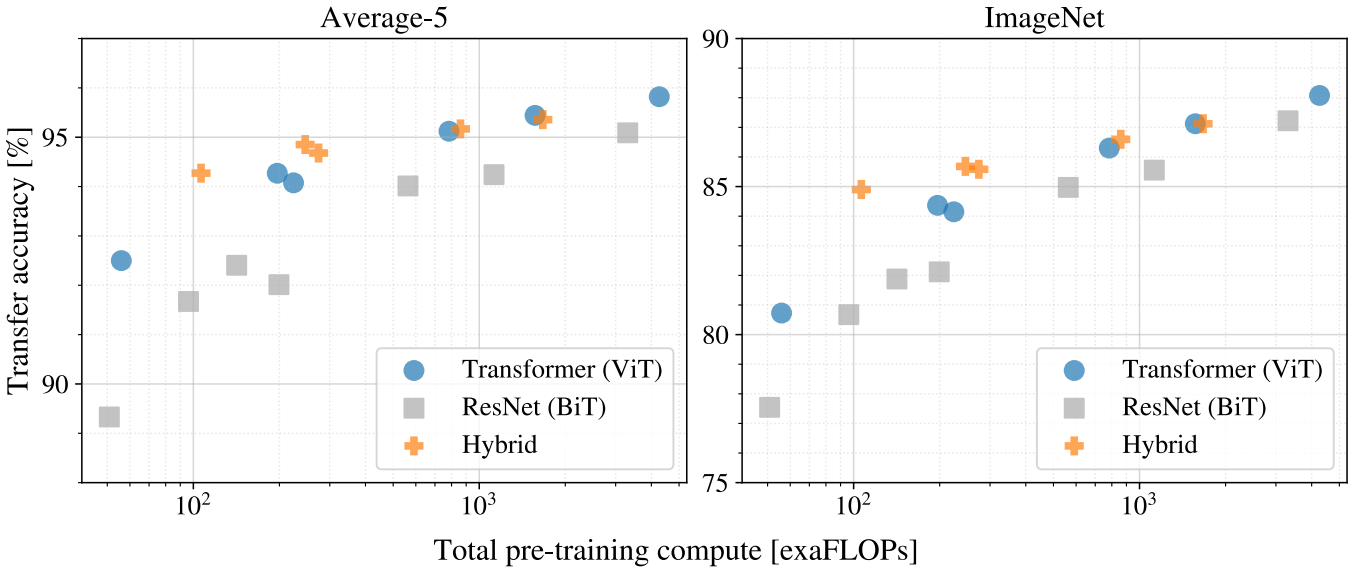
# The best of both worlds

- Popular architectures combine CNN and attention.
- Here Hybrid = BiT + ViT

  https://huggingface.co/google/vit-hybrid-base-bit-384

➔ integrates a BiT (Big Transfer) ResNet backbone with a ViT.

- CNN extracts features, used as initial "tokens" for the Transformer.




- Hybrids improve upon pure Transformers for smaller model sizes.
- Gap vanishes for larger models.

# Why do we need vision transformers (ViT)?

- **Attention**: New paradigm extended from language to computer vision. The goal: learn strong meaningful representations with global and local context.

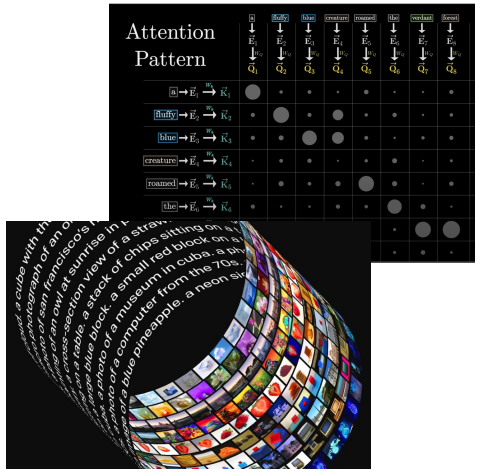
- **Language and vision** are the two big domains with successful DL applications. Merging them using one architecture allows for richer options ➔ multi-modal tasks (involving text, images, audio, video…)

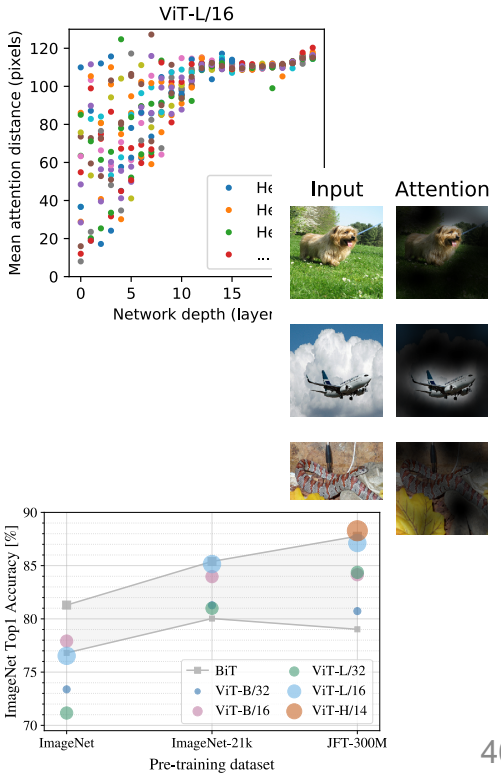
- **Allow more freedom in feature discovery:** ML methods provide diffferent levels of inductive bias during learning: handcrafted features ➔ many, CNNs➔ fewer, transformers ➔ yet fewer. It is the most data driven architecture.

- **Built-in interpretability** mechanism = attention maps.

- **Outperforms CNNs** under certain conditions (pre-trained on big enough data).

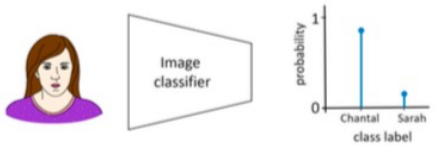
- **Combinable with CNNs.**

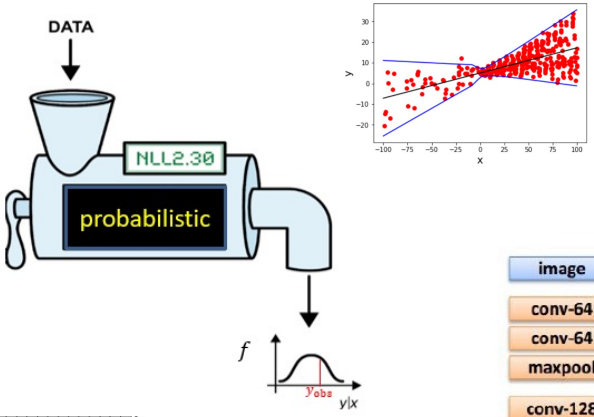


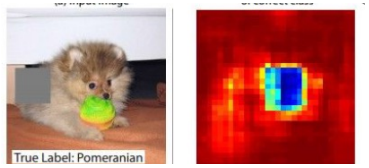



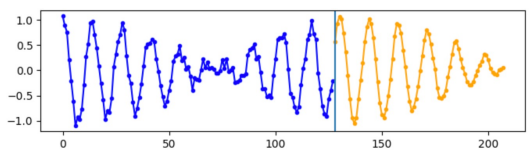
# Summary CAS Deep Learning

1. Predict (class, age…) with NNs.
2. NLL loss to predict a probabilistic outcome.
3. CNNs for image prediction tasks.
4. Pretrained CNNs for efficient representation learning.
5. xAI (e.g occlusion) for interpretability.
6. DL for more tasks (representation learning, reinforcement learning, GenAI).
7. Vision Transformers.

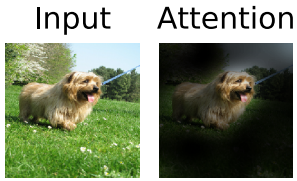






# Summary